

The Enzyme-Constrained Thermodynamic Flux Module

What it computes, what it is fed,
and what has been measured

Michael Volk

September 4, 2026

Abstract

The flux module is a differentiable layer that maps a perturbed gene representation to a flux vector over yeast-GEM 9.0.2, subject to stoichiometry, an enzyme-capacity bound and a thermodynamic sign constraint. It is neither a Type I nor a Type II virtual instrument as those were defined: its output is a latent physical state rather than a measured phenotype, and the phenotypes are read off that state.

Its distinguishing property is that three of its inputs are not learned. The turnover numbers, the Michaelis constants and the formation energies are tabulated, hash-pinned, and attributable to the weights or model version that produced them. That is what makes the layer a reconstruction of a metabolic network rather than a parameterization fitted to one organism, and it is why the parameter tables were built across five independent predictors instead of one.

This document consolidates what was scattered across the 026 and 027 notes: the formulation as implemented, the three parameter tables and the sharp distinction between those wired in and those merely built, the ablation registry, and every measured number. Two corrections to earlier claims are recorded rather than quietly folded in. The ChemAxon licensing blocker does not apply to the betaxanthin pathway, whose intermediates are already in the compound cache. And the reaction-coverage shortfall is not a mapping gap: an audit that recovers every identifier route available gains 0.58 percentage points, because 375 metabolites match a cache record that carries no structure and are counted as resolved while returning no energy.

The training runs that test whether the module helps are on Delta as this is written, and the preliminary section reports only what has actually been observed.

Section status: ✓ ready ■ author-reviewed, keep stable ✗ not done, agents may edit freely

Contents

1	What the module computes ✗	3
1.1	Where it sits ✗	3
1.2	A program cannot learn, which is the point ✗	3
1.3	The relaxation, term by term ✗	3
1.4	Free against anchored thermodynamics ✗	4
2	The three parameter tables ✗	4
2.1	What was there before, and what is there now ✗	4
2.2	Which tables are actually wired in ✗	4
2.3	Why five predictors and not one ✗	5
3	Thermodynamic coverage, and two corrections ✗	5
3.1	The recomputation ✗	5
3.2	Correction: the shortfall is not a mapping gap ✗	5
3.3	A correctness bug in the shipped build, found by the same guard ✗	6
3.4	Correction: the ChemAxon blocker does not apply here ✗	6
4	The ablation registry ✗	6
4.1	What each arm turns on ✗	6
4.2	The control that is missing, and why it matters ✗	7
4.3	The permutation nulls ✗	7
5	What has been measured ✗	7
5.1	The 026 sweep, and why it decided nothing ✗	7
5.2	The defect that outranks all three ✗	8
5.3	The baselines the module has to beat ✗	8
5.4	The noise floor, which sets the replicate count ✗	8
6	The run now on Delta ✗	8

6.1	What was launched ✖	8
6.2	Three faults found and fixed before submitting ✖	9
6.3	Three couplings the backbone change forced ✖	9
6.4	Preliminary training results ✖	9

1 What the module computes ✗

1.1 Where it sits ✗

The Equivariant Cell Graph Transformer separates its instruments by what they map between. A **Type I** instrument is representation to representation: the perturbation operators T_ψ^{del} , T_ψ^{OE} , T_ψ^{KD} act inside the gene embedding space and hand on H_{pert} . A **Type II** instrument is representation to output: a readout that emits a measured phenotype.

The flux module is neither, and the reason is worth stating precisely rather than settling by convention. By the letter of the taxonomy it is Type I, since its output feeds further computation rather than a loss. But every existing Type I instrument acts *within* the gene index space at dimension d and preserves equivariance over genes, whereas this one leaves gene index space entirely for reaction space $\mathbb{R}^{4131}$ under a fixed, externally supplied stoichiometry, and its parameters are tabulated physical constants rather than learned weights. It is a physically identified decoder, and its output ν is a latent physical state rather than a phenotype: the phenotypes are read *off* it.

1.2 A program cannot learn, which is the point ✗

The constraint-based tradition solves a linear program per condition. A linear program has no use for a phenotype measurement, so it cannot be improved by one. This layer inverts that: every constraint is either enforced exactly by the parameterization or relaxed to a smooth penalty, so the flux is *conditioned* on roughly 10^7 fitness records, 4,735 betaxanthin measurements and 4,678 amino-acid profiles. **Nothing is enforced by a binary variable**, which is what keeps the whole object differentiable and lets a batch of perturbations cost the same forward pass as a single one.

1.3 The relaxation, term by term ✗

Table 1: Each constraint of an enzyme-constrained thermodynamic model, and how this layer realizes it. “Exact” means the parameterization cannot violate it; “soft” means a penalty that the optimizer trades against the data term.

Constraint	Conventional form	Here
box and directionality	$lb_j \leq \nu_j \leq ub_j$	exact , via sigmoid
enzyme capacity	$\nu_j \leq k_{cat} E$	exact , folded into the box
mass balance	$S\nu = 0$	soft, scale-relative
second law	$\Delta_r G_j \nu_j < 0$ plus a binary	soft hinge, no binary
Gibbs dissipation	$g_{\text{diss}} \leq g_{\text{lim}}$	soft hinge
protein budget	$\sum_g MW_g E_g \leq P_{\text{avail}}$	soft hinge

From representation to gene availability. Each gene’s embedding passes through a learned gate, $\gamma_g = \sigma(w^\top h_g)$, so a gene is not merely present or absent. A deletion pins $\gamma_g = 0$; everything else is free to take an intermediate value, which is what leaves room for dosage, hypomorphic alleles and overexpression rather than forcing the layer to treat every perturbation as a knockout.

Folding up the gene-protein-reaction rule. Availability is folded to the reaction through the GPR: a **softmax** over the subunits of a complex, since a complex is limited by its scarcest member, and a **sum** over isozymes, since they substitute for one another. The softmax temperature is a real parameter, not a detail: too soft and a complex behaves like a mean over its subunits, which loses exactly the limiting-subunit behavior the softmax was chosen for.

The box, which is where capacity lives. Flux is parameterized as

$$\nu_j = lb_j + (ub_j - lb_j) \sigma(z_j), \quad |\nu_j| \leq k_{cat,j} E_{g(j)}, \quad (1)$$

so the bounds and the capacity bound hold by construction and cannot be violated no matter what the network does. Turnover enters in units of h^{-1} ($k_{cat} [\text{s}^{-1}] \times 3600$), which is what fixes the units of the enzyme abundance E . The exactness budget is spent here rather than on $S\nu = 0$, and that is a deliberate choice: bounds are exact and mass balance is soft, not the reverse.

The second law, which is the load-bearing relaxation.

$$C_{\text{th}} = \frac{1}{|\mathcal{J}|} \sum_{j \in \mathcal{J}} \text{relu}(\nu_j \Delta_r G_j + \epsilon), \quad (2)$$

with $\mathcal{J}$ excluding exchange reactions, the biomass reaction and water transport, which cross the system boundary or are lumped and so carry no meaningful reaction energy. The ϵ is not decoration. The conventional big-M pair admits $\nu_j = \Delta_r G_j = 0$ under either value of its binary, so the strict inequality is never actually realized, and a smooth version has to name its tolerance rather than inherit a fiction.

Loop-freedom comes free, and that is the trick. Because $\Delta_r G$ is a difference of potentials it sums to zero around any cycle, so no cycle can run downhill at every step. Requiring $\nu_j \Delta_r G_j \leq 0$ therefore forbids internal loops with no integer variable at all, which is precisely the property that makes loopless flux balance analysis a mixed-integer program.

1.4 Free against anchored thermodynamics ✖

The in-cell reaction energy decomposes into three terms,

$$\Delta_r G_j = \underbrace{\Delta_r G_j^{to}}_{\text{tabulated}} + \underbrace{RT \sum_{i \neq H^+} S_{ij} \ln c_i}_{\text{concentration}} + \underbrace{\Delta_r G_j^t}_{\text{transport}}. \quad (3)$$

The distinction between the two thermodynamic arms is exactly which of these is supplied. In the **free** arm the potential is learned, so the layer gets loop-freedom and nothing else; the constraint is structural. In the **anchored** arm the standard term is read from the tabulated formation energies, so the sign of every reaction is pinned by measured chemistry rather than by whatever the optimizer finds convenient. That difference is the experiment: structural loop-freedom is a real constraint on its own, and the question is whether the chemistry adds anything to it.

2 The three parameter tables ✖

2.1 What was there before, and what is there now ✖

The module needs three tables and had almost none of them. Measured k_{cat} covered 148 of 3,728 catalytic units, 4.0 percent, with the other 96 percent running on a single flat default of 10.3 s^{-1} . K_M had no table at all: the parameter existed as an enum member and a protocol flag with nothing behind either. The thermodynamics came from a shipped file of point values with no covariance, usable only for this organism.

Six kinetic predictors were mirrored into `$DATA_ROOT/models/kinetics/<name>/` with per-file `sha256`, retrieval commands and git commits, and all six now run over the enzyme-substrate pairs of yeast-GEM 9.0.2, with protein sequences taken from the genome object rather than from the table each model bundles.

Table 2: Every parameter table now built, with the exponentiation base established by validating each predictor against its own authors’ published output. Coverage is of the GEM’s 3,728 catalytic units.

Predictor	Parameter	Rows	Units	Coverage	Median
DLKcat	k_{cat}	7,351	3,552	95.3%	4.012 s^{-1}
UniKP	k_{cat}	7,351	3,552	95.3%	4.050 s^{-1}
EITLEM	k_{cat}	7,456	3,552	95.3%	3.981 s^{-1}
TurNuP	k_{cat}	5,143	2,056	55.2%	10.819 s^{-1}
DeepEnzyme	k_{cat}	7,300	3,546	95.1%	4.167 s^{-1}
UniKP	K_M	7,351	3,552	95.3%	0.0796 mM
EITLEM	K_M	7,456	3,552	95.3%	0.0793 mM
Boost_KM	K_M	7,157	3,468	93.0%	0.1410 mM

Two of these models exponentiate base 2 and four base 10. DLKcat and DeepEnzyme are $\log_2$; UniKP, EITLEM, TurNuP and Boost_KM are $\log_{10}$. Reading either wrong yields turnover numbers that still look entirely credible, which is why each was validated against something its own authors published before its values were believed.

2.2 Which tables are actually wired in ✖

This distinction decides how any result from the module should be read, and it is easy to lose.

Table 3: What the layer consumes today. A table that is built but not consumed contributes nothing to a measured result and must not be described as if it did.

Table	Status	How it enters, or why it does not
k_{cat}	wired , but from the Open Enzyme Database, not the predictors	the capacity bound $ \nu_j \leq k_{cat,j} E_{g(j)}$. <code>resolve_kcat_table</code> reads the OED mirror, so the run uses 4.0 percent measured values and a default for the rest. The predicted tables are not yet plumbed to this call
$\Delta_f G'^o$	wired in the anchored arm	the standard term of $\Delta_r G_j$, hence the second-law hinge and the dissipation bound
K_M	built, not wired	the saturation term is drawn as inactive in the module diagram and is inactive in the code. K_M contributes nothing to any result reported here

The consequence is worth stating plainly: the current runs test the module’s *structure and thermodynamics*, not its predicted kinetics. Wiring the predictor registry into `resolve_kcat_table` is the next change, and it is the one that turns the five kcat tables from an audit into an input.

2.3 Why five predictors and not one ✗

Re-keying the comparison from the enzyme-substrate pair to the **gene**, which is the unit the capacity bound reads, gives the number that decides how a predicted table should be used.

Table 4: Across-predictor spread per gene against the dynamic range of the quantity itself, in decades, over genes every predictor covers.

Quantity	Genes	Median spread	Consensus p10–p90	Ratio
k_{cat}	980	1.23	0.94	1.32
K_M	1,150	0.70	1.34	0.52

For k_{cat} the disagreement between predictors exceeds the quantity being predicted. The median gene moves 1.23 decades between the highest and lowest predictor, a factor of 17, and 67 percent of genes move by more than a full decade, against a 10th-to-90th-percentile spread across genes of 0.94 decades. Choosing a different published predictor moves one gene’s turnover number further than real enzymes differ from one another. Gene-level Spearman between k_{cat} predictors runs 0.04 to 0.48.

For K_M it does not. The same construction gives 0.70 against 1.34, and Boost_KM agrees with UniKP at a gene-level Spearman of 0.75, higher than any pair of k_{cat} predictors reaches. Both are statements about agreement rather than accuracy, since almost none of these pairs has a measured value.

That is why the registry in `torchcell/datasets/kinetics/` exists and why a config string selects the predictor: predictor choice is a hyperparameter with a measured effect size, not a settled detail.

3 Thermodynamic coverage, and two corrections ✗

3.1 The recomputation ✗

Formation energies were recomputed from eQuilibrator’s component contribution, which returns a covariance and applies to any organism because it works from structure rather than from a curated per-organism file. Summing the resulting table reproduces eQuilibrator’s own reaction call to 9.1×10^{-13} kJ mol^{−1}, so the table is arithmetically correct. On the chemical reactions both tables cover it agrees with the shipped file to a median of 10.11 kJ mol^{−1}.

The gain is 785 reactions with a real standard error, median 2.30 kJ mol^{−1}, and the rank-590 covariance factor the layer had recorded as unavailable. The cost is reaction coverage: 42.5 percent against the shipped file’s 77.7.

3.2 Correction: the shortfall is not a mapping gap ✗

The obvious reading of that 42.5 percent is that identifiers are failing to resolve, and that better mapping would close it. An audit that implements every available recovery route measures otherwise.

There are two loss channels, and the summary reported only one. The 332 “unresolved” metabolites are the accession-match failures. A further **375 metabolites match a cache record that carries no structure and no group vector**, so `standard_dg_formation` returns nothing while the tier histogram counts them as resolved. Total

unpriced is **707**, not 332. The baseline is 2,099 of 2,806 metabolites, 74.80 percent, and 1,755 of 4,131 reactions, 42.48 percent.

Table 5: Every identifier route available, applied in turn. Reaction coverage is the binding number because a reaction needs every participant priced.

Step	Metabolites priced	Reactions	Δ reactions
baseline	2,099 (74.80%)	1,755 (42.48%)	—
extra identifier namespaces	2,102	1,764 (42.70%)	+9
MetaNetX cross-reference table	2,113	1,770 (42.85%)	+6
InChIKey structure lookup	2,127 (75.80%)	1,779 (43.06%)	+9

Total recovery is 28 metabolites and 24 reactions, 0.58 percentage points. The extra namespaces are worth exactly one chemical, hydrogen sulfide. The census explains why: the model carries chebi on 2,402 metabolites and metanetx on 2,251, but biocyc on 29, seed on 16, reactome on 2, and no hmdb, lipidmaps or swisslipid at all. Of the 332 unmatched, **181 of the 192 distinct chemicals carry no chemical annotation whatsoever**, and the class breakdown is overwhelmingly lipid: phosphatidylinositol 135, cardiolipin 62, triacylglycerol 34, phosphatidate 25.

Of the 679 still unpriced, 249 (36.7 percent) are a real ceiling: 185 carry a placeholder formula with an R, X or * so no definite structure exists, and 64 are known to eQuilibrator but have no group decomposition. The other 430 (63.3 percent) have a definite formula and no usable structure anywhere, which is a structure-curation gap rather than a limit of the method.

3.3 A correctness bug in the shipped build, found by the same guard ✗

The InChIKey route initially reported a gain six times larger, and it was wrong. MetaNetX writes [*] dummy atoms for polymers and conjugates; RDKit returns an *empty* InChIKey for these rather than failing; an empty key reaches the cache as LIKE '%' and matches the entire compound table. *Ala-tRNA(Ala)* was being priced as water.

The guard that fixes it, comparing heavy-atom composition against the GEM formula with hydrogen excluded because protonation state legitimately differs, then finds the same class of error in the build already in use: **31 metabolites currently carry a formation energy from a compound whose heavy-atom composition disagrees with the GEM formula**, and 14 of the 1,755 covered reactions touch one. Every dolichol species is matched to a C25 pentaprenyl against the GEM's C20 tetraprenyl; **glycogen** is matched to a C24 tetramer against a C6 monomer; **protein**, **carbohydrate** and **sterols** are matched to concrete molecules.

This does not raise coverage and it is worth more than coverage: it removes 31 wrong numbers. It is not yet applied to `resolve_compound`, and it should be.

3.4 Correction: the ChemAxon blocker does not apply here ✗

An earlier account of this work stated that pricing a novel compound requires ChemAxon's `cxcalc` for pKa estimation, that no license is available, and that the betaxanthin intermediates therefore cannot be priced. The first two clauses are true and the conclusion is false: **all of the pathway's intermediates are already in the compound cache**, so nothing about betaxanthin needs a new compound created.

For a genuinely novel compound the constraint has also loosened. Upstream `equilibrator-assets` merged support on 2026-08-15 for creating compounds without ChemAxon when pKa values are supplied, via `pkas_provided_externally` and `specified_pkas`. That is master-only; the pinned release 0.6.0 does not contain it. What the interface needs is an unannotated list of macroscopic pKa floats, which is a weaker requirement than the site-annotated micro-pKa the harder tools produce.

4 The ablation registry ✗

4.1 What each arm turns on ✗

The registry is one dictionary, **ARMS** in `experiments/026-metabolism-flux/scripts/train_flux.py`, and it is the single source of truth for what the layer does in any run. An experiment names an arm; it never redefines one. That is what keeps a difference between two arms attributable to the module rather than to a hyperparameter that moved alongside it.

Table 6: The registry as implemented. Each row is a strict addition to the one above it except **nullspace**, which changes the parameterization rather than the physics.

Arm	Flux	Thermo	Capacity, budget, dissipation	Isolates
pooled	off	—	—	the no-module control
flux_off	on	OFF	off	routing through S alone
flux_free	on	FREE	off	structural loop-freedom
flux_anchored	on	ANCHORED	on	the treatment
flux_nullspace	on	ANCHORED	on	the exactness budget

The ladder is deliberate. **flux_off** routes the representation through the stoichiometric network with nothing but mass balance and the GPR, so it asks whether the network’s topology alone carries signal. **flux_free** adds a learned potential, which buys loop-freedom and no chemistry. **flux_anchored** replaces the learned potential with tabulated formation energies and switches on the enzyme capacity bound, the protein budget and the dissipation limit. **flux_nullspace** keeps the anchored physics but spends the exactness budget on $S\nu = 0$ instead of on the box, which is the one arm that tests the parameterization choice rather than the model.

4.2 The control that is missing, and why it matters ✗

Every arm above changes the module’s *physics* together with its *parameter count*. So a gap between **flux_anchored** and **pooled** is consistent with two different explanations: the tabulated chemistry is informative, or the extra structure acts as a capacity constraint that would help with any numbers in it.

The arm that separates those is **flux_shuffled**: the full anchored module with k_{cat} **permuted across reactions**, holding architecture, parameter count and optimization fixed and destroying only the correspondence between a reaction and its turnover number. If **flux_anchored** and **flux_shuffled** land together, the kinetic tables are decoration.

It is **not in the registry and not in the current run**. Adding it is a code change that was not worth making untested, and it is the first item of the next round.

4.3 The permutation nulls ✗

Two arms exist only to be compared against: **null_pooled** and **null_anchored** train on **permuted labels** while validating on real ones. They matter because of what 026 measured. Against zero, its flux-versus-pooled gap of +0.0735 is 7.7 standard errors. Against its own permuted-label gap of +0.0302, the excess is +0.0433 at 1.9 standard errors. More than 40 percent of “the flux module helps” survived destroying the labels.

So the decision rule is a difference of differences,

$$(\text{flux} - \text{pooled}) - (\text{null flux} - \text{null pooled}), \quad (4)$$

and a gap measured against zero is not evidence.

5 What has been measured ✗

5.1 The 026 sweep, and why it decided nothing ✗

Table 7: The 026 sweep on betaxanthin. The statistic is validation Pearson scored as a *maximum over epochs*, which is an upward-biased order statistic, and the last column is the mean of the final five epochs, which is not.

Arm	Seeds	Peak mean	s.e.m.	Peak epoch	Last-5 mean
flux_anchored	10	0.1151	0.0095	17	0.0057
flux_free	10	0.1020	0.0115	12	0.0212
flux_off	10	0.0837	0.0112	10	0.0044
pooled	10	0.0416	0.0098	4	−0.0041
flux_nullspace	10	0.0381	0.0114	0	0.0000

The ordering looks like the ladder the design predicted. It is not evidence, for three measured reasons.

No arm clears its own null. A label-permutation null over 24 runs has peak mean 0.0962, standard deviation 0.0441, and a 95th percentile of **0.1545**. No arm reaches that percentile. The smallest p across all 22 configurations tested is 0.24, and the single highest peak anywhere in the sweep, 0.2110, belongs to a *null* run.

The last-5 column is the honest one. Every arm’s peak sits far above its own end-of-training value, and three of five end negative. A maximum over 30 epochs of a statistic whose analytic null width at $n_{\text{val}} \approx 340$ is 0.0546 inflates that width empirically to 0.0962, a factor of 1.8.

The split moved with the seed. 026 re-rolled the train/validation partition per seed, so seed varied the nuisance and the signal together. Its paired arm gap has an across-seed standard deviation of 0.0303 with real labels and 0.0722 with permuted ones, both inflated by the partition moving underneath the contrast.

5.2 The defect that outranks all three ✗

Every betaxanthin run in this repository falls into one of two architecture families, and the family accounts for nearly all of the variance between runs.

Table 8: Measured validation Pearson by backbone. The best betaxanthin result ever obtained is 0.43399, GilaHyper job 1344, config `gh_metabolism_002`, seed 42, peak at epoch 39 of 80.

Family	Configuration	Measured
weak	2 graphs, learnable embeddings, MSE, hidden 32, 4 heads	0.04 to 0.16
strong	9 graphs, <code>prot_T5_all</code> , no learnable embedding, CRPS, hidden 90, 9 heads	0.32 to 0.43

`train_flux.py` builds the **weak** family. So 026 compared a flux module against a CGT running roughly five times below its own demonstrated ceiling, and its `flux_off` arm at 0.0837 is the correct control *for that sweep* rather than a CGT baseline. Nothing in 026 tested whether the module helps a CGT that works, which is what the current run exists to do.

5.3 The baselines the module has to beat ✗

Table 9: Every measured comparator, with the statistic named. FCL is Merzbacher’s Flux Cone Learning, a three-class classifier rather than a regressor, scored on its own 639-gene test split.

Comparator	Statistic	Value
CGT, strong backbone	max val Pearson, $n_{\text{val}} = 340$	0.43399
FCL RandomForest_Resampled	gene-level accuracy (majority 0.674)	0.700
FCL	Matthews correlation	0.232
FCL	Pearson against the raw screen	0.2435
FCL	Spearman against the raw screen	0.0391
fully-connected / MLP baseline	not measured; does not exist	–

There is no MLP baseline for betaxanthin anywhere in the repository. If one is wanted it must be built, and no number should be quoted for it until it is.

5.4 The noise floor, which sets the replicate count ✗

Four independent measurements of the across-seed standard deviation on this endpoint: 0.0302 from hyperparameter-matched reruns, 0.0355 from ten fresh seeds of `flux_off`, 0.0299 from ten seeds of `flux_anchored`, and 0.0337 from a three-seed 019 arm. The analytic single-correlation width at $n_{\text{val}} \approx 340$ is 0.0546.

Two consequences were computed rather than asserted. Selection inflation over 36 Optuna trials is 0.064, which puts the bias-corrected floor of the 0.4301 best at **0.3662**. And reaching an effective standard error of 0.010 needs **ten replicates per arm**, which is what sets the current run’s twelve.

6 The run now on Delta ✗

6.1 What was launched ✗

Job array 21797666_[0-8] on `bbub-delta-gpu`, partition `gpuA40x4`: nine nodes, four A40 per node, **two runs per GPU**, so 72 workers. Six arms divide 72 into twelve seed-groups, giving twelve seeds per arm, above the ten the noise floor requires. Each worker owns its own Optuna database, which is what avoids the race that previously had several workers claiming one trial and silently turning four Delta cells into duplicates.

The primary endpoint is Spearman on the pinned Merzbacher 639-gene test split at an epoch selected on validation, so the selection set and the reporting set are disjoint. The decision rule is the difference of differences in Section 4.

Table 10: Every environment fact confirmed on the node rather than assumed, since the repository’s own files disagree about several of them.

Fact	Value
DATA_ROOT	/scratch/bbub/mjvolk3/torchcell. The Delta .env still says /work/hdd, which is wrong; pointing at a root with no build makes the dataset attempt a Neo4j rebuild on a compute node
interpreter	/work/hdd/bbub/miniconda3/envs/torchcell/bin/python, called directly, no conda activate and no container
GPU	NVIDIA A40, 46,068 MiB, four per node
NUM_WORKERS	0, forced. Spawned workers re-import off Lustre and stall
W&B	online, entity zhao-group, project torchcell_027_bxfx

6.2 Three faults found and fixed before submitting ✗

The Open Enzyme Database mirror was absent from Delta. `resolve_kcat_table` catches the resulting `FileNotFoundError` and returns an empty record list, which replaces every measured k_{cat} with the organism default. The enzyme-constrained arm would have run with no enzyme constraints and reported a plausible number. The mirror is 516 kB and was synced; the coverage gate exists to abort exactly this case, and it can only fire if there is a mirror to measure.

WANDB_ENTITY was unset, so runs would have landed under whatever default entity the node carried while 026’s are under `zhao-group`, leaving the arms unjoinable at analysis time.

OMP_NUM_THREADS was unset, so eight workers on a 32-CPU allocation of a 64-core node would each have defaulted to the node’s core count.

6.3 Three couplings the backbone change forced ✗

Moving 027 onto the strong backbone broke three things, each found by running a smoke test rather than by reading the code, and each silent in a different way.

1. The perturbation head defaults to eight attention heads and 90 is not divisible by eight, so its cross-attention refuses to build. 019 uses six.
2. The model reads `cell_graph["gene"].x` only when it is told it has precomputed embeddings. With the learnable table disabled and `node_embeddings` unset it finds neither and raises. It also indexes the embedding dataset to infer its width, so it needs the objects rather than their names.
3. Dropping 026’s `build_dataset` dropped its module globals with it.

A two-epoch local smoke run on the corrected backbone moved loss 1.0586 to 1.0088 and validation Pearson -0.0214 to $+0.1055$, and computed the pinned test Spearman at $+0.0261$ on $n = 629$. Two epochs is not a result. What it establishes is that the nine-graph `prot_T5` backbone builds, trains and scores end to end.

6.4 Preliminary training results ✗

None yet. At the time of writing, four of the nine array tasks are running and five are queued behind them on resources. No worker has emitted a training epoch, so there is no validation Pearson to compare and none is quoted here.

The delay is understood rather than mysterious. The launcher runs a sequential `-create-only` pass per worker before any GPU work, both to create each worker’s Optuna database and to pay the cold-import cost once so the Lustre client cache is warm for the rest. That first import is reading at roughly 60 kB s^{-1} : the process sits in `ptlrpc_set_wait`, the Lustre RPC wait, at about one percent CPU, having read 90 MB after twenty minutes. It is progressing, and a 20-hour allocation absorbs an hour of cache warming, but the observation is worth recording because the same cost has previously been mistaken for a hang and has killed canaries with short wall limits.

This section will carry the arm comparison once the sweep produces one. The comparison that matters is not a single validation Pearson but the difference of differences against the permuted-label nulls, and that needs the seeds to finish.